

Workflow do Sistema

Analise Automatizada de Diagramas de Arquitetura com IA

Projeto	Hackathon FIAP - Time IADT
Versao	1.0
Data	2026-04-16
Stack	Python, FastAPI, Celery, Redis, PostgreSQL+pgvector, LangChain, OpenAI gpt-4o
Padroes	DDD + Arquitetura Hexagonal (Ports & Adapters)
Servicos	ia-service (8000), report-api (8001), streamlit-app (8501), celery-worker, redis, pgvector

Sumario

1.	Visao Geral do Sistema	<i>cap. 1</i>
2.	Workflow Principal - Visao de Alto Nivel	<i>cap. 2</i>
3.	Workflow Detalhado - Caminho A (Streamlit)	<i>cap. 3</i>
4.	Workflow Detalhado - Caminho B (AWS SQS)	<i>cap. 4</i>
5.	Pipeline de IA - As 5 Etapas + Guardrails	<i>cap. 5</i>
6.	Convergencia e Tratamento de Falhas	<i>cap. 6</i>
7.	Persistencia e Observabilidade	<i>cap. 7</i>
8.	Workflow de Consulta de Relatorio	<i>cap. 8</i>
9.	Resumo Visual do Workflow	<i>cap. 9</i>
10.	Glossario	<i>cap. 10</i>

1. Visao Geral do Sistema

1.1 O que o sistema faz

O sistema automatiza a analise de diagramas de arquitetura de software. Recebe uma imagem (PNG, JPG, JPEG, GIF, WEBP) ou PDF e devolve um relatorio tecnico estruturado contendo:

- **Componentes** identificados no diagrama
- **Relacionamentos** entre componentes
- **Padroes arquiteturais** reconhecidos
- **Riscos** em 6 categorias (SPOF, Seguranca, Escalabilidade, Acoplamento, Observabilidade, Resiliencia) com severidade ALTO/MEDIO/BAIXO
- **Recomendacoes** de melhoria (marcadas com [RAG] quando vindas do historico)
- **Sumario executivo**
- **Score de QA** (0 a 1) validando o proprio relatorio

1.2 Atores e componentes

Ator / Componente	Papel
Usuario final	faz upload do diagrama pelo Streamlit
Time SOAT	publica mensagens no SQS com URL do S3
ia-service	orquestrador do pipeline
Celery Worker	executa o pipeline em background
Redis	broker do Celery + pub/sub de eventos SSE
PostgreSQL + pgvector	persiste analises, relatorios e embeddings
OpenAI (gpt-4o)	LLM Vision (extracao) + LLM Text (relatorio, QA)
report-api	API read-only para consulta de relatorios

1.3 Mapa de componentes

Arquitetura de Alto Nivel

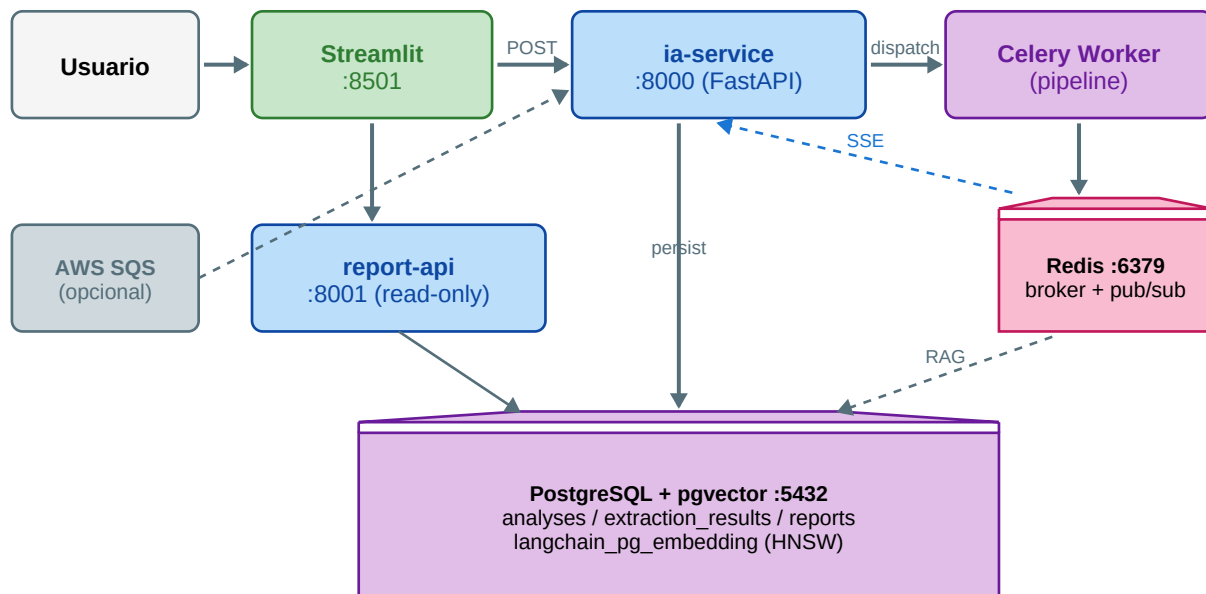
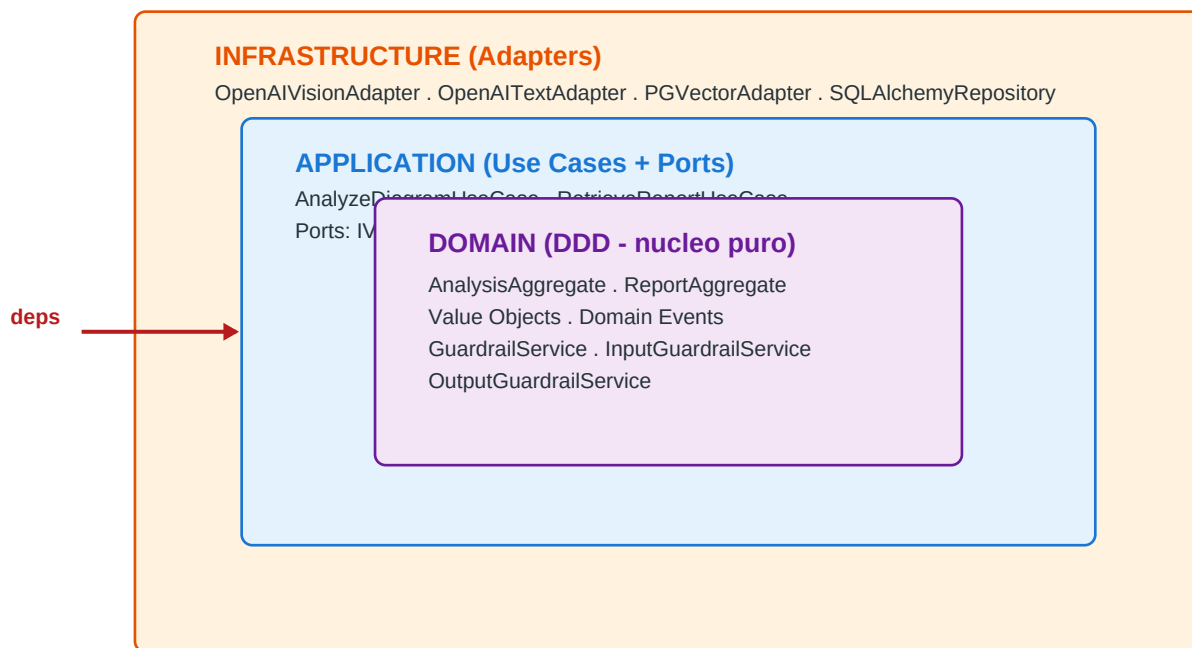


Figura: Visão de alto nível dos serviços e fluxos principais

1.4 Arquitetura em camadas

O **ia-service** adota **Arquitetura Hexagonal** (Ports & Adapters). As dependências apontam sempre para dentro: **Infrastructure** depende de **Application**, que depende de **Domain**. O núcleo do domínio não conhece nada externo.

Arquitetura Hexagonal (Ports & Adapters)



Regra: dependencias apontam SEMPRE para dentro. Domain nao importa nada externo.

Figura: Camadas - dependencias sempre apontam para dentro

1.5 Fluxos de entrada (3 caminhos, mesmo pipeline)

- **Caminho A - Upload via Streamlit:** Streamlit -> POST /analyze/async -> Celery -> pipeline -> SSE
- **Caminho B - AWS SQS:** SOAT -> SQS -> Consumer -> S3 download -> pipeline -> Webhook
- **Caminho C - API direta (testes):** cliente -> POST /analyze (sincrono) -> pipeline -> JSON

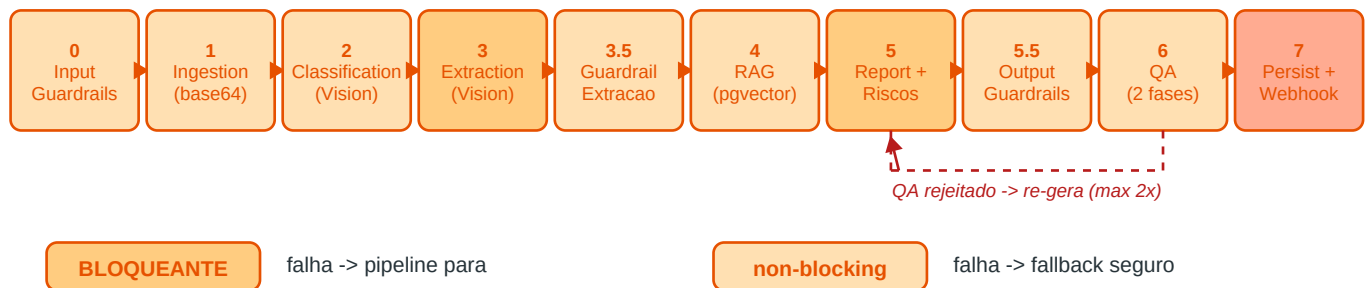
Todos convergem para `AnalyzeDiagramUseCase.execute()` (`application/use_cases/analyze_diagram.py`), garantindo consistencia de comportamento.

2. Workflow Principal - Visao de Alto Nivel

2.1 Fluxo ponta-a-ponta

O fluxo parte do upload do diagrama, passa pelas 8 etapas do pipeline de IA e termina com o relatorio persistido no banco e entregue ao solicitante (via SSE ou webhook).

Pipeline de IA - 8 Etapas Sequenciais



ExtractionResult = Ground Truth

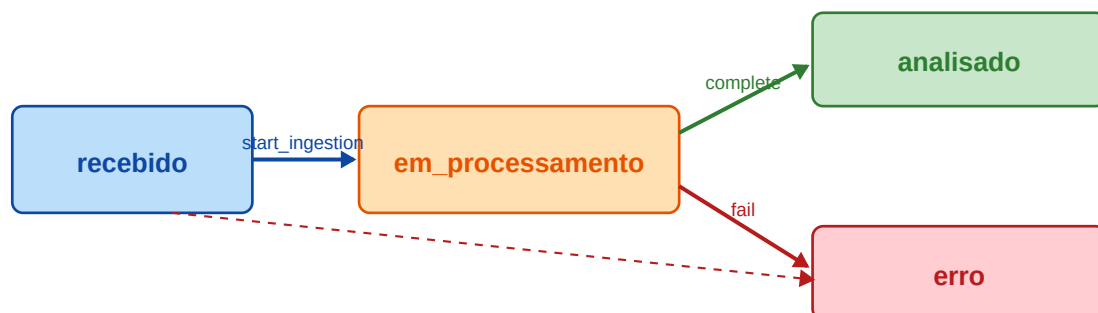
Todas as etapas seguintes sao validadas contra ele (anti-alucinacao)

Figura: Pipeline de IA com 8 etapas e loop de refinamento do QA

2.2 Estados de uma analise

Toda analise tem um ciclo de vida com 4 estados possiveis. A maquina de estados e protegida pelo **AnalysisAggregate** (DDD), que garante invariantes e emite **Domain Events** a cada transicao.

Maquina de Estados da Analise



Invariante: apenas 'recebido' pode transitar para 'em_processamento'. Qualquer estado pode ir para 'erro'.

Figura: Maquina de estados do AnalysisAggregate

3. Workflow Detalhado - Caminho A (Streamlit)

Este é o fluxo principal. Use este capítulo como referência para rastrear e debugar. Cada passo aqui é numerado em ordem temporal.

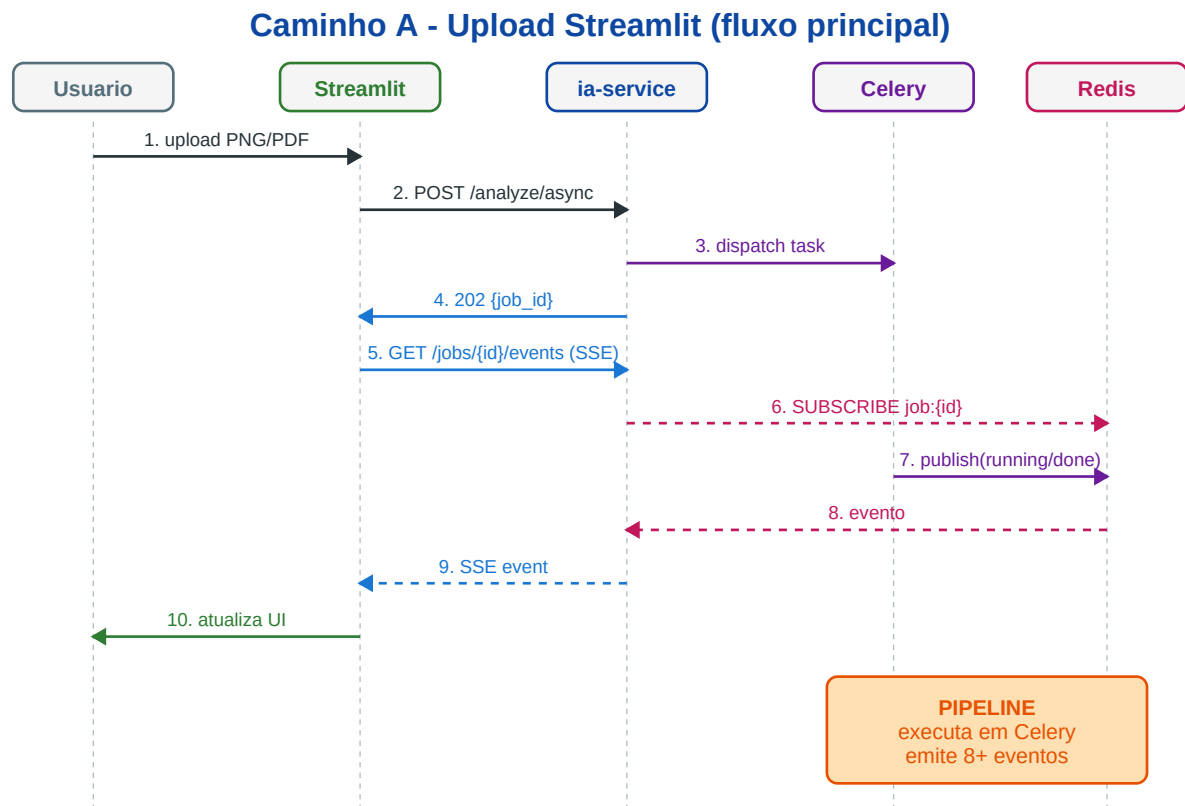


Figura: Sequencia das mensagens entre Usuario, Streamlit, ia-service, Celery e Redis

Passo 1 - Upload do diagrama

O usuário arrasta o arquivo na UI do Streamlit (:8501). O Streamlit lê os bytes e mostra preview se for imagem. Ao clicar em **Analisar Diagrama**, monta um multipart/form-data e dispara a requisição.

Passo 2 - POST /analyze/async

O Streamlit envia o arquivo ao ia-service em modo assíncrono. O ia-service responde em menos de 100ms com HTTP 202 Accepted e um job_id (UUID) usado para tracking via Redis.

Passo 3 - ia-service cria job e dispara task Celery

O ia-service gera um job_id, serializa o arquivo como file_hex (hex preserva bytes em JSON) e chama `analyze_diagram_task.delay(file_hex, file_name, job_id)`. O Celery publica a task no Redis (broker).

Passo 4 - Streamlit abre stream SSE

Imediatamente após receber o job_id, o Streamlit inicia `GET /jobs/{job_id}/events`. Do lado do ia-service, o endpoint faz `SUBSCRIBE` no canal Redis `job:{id}` e repassa cada mensagem como evento SSE.

Passo 5 - Celery Worker consome a task

O Worker (concurrency=2) faz polling no Redis, pega a task, decodifica `file_hex` -> `bytes`, abre a sessao no banco e monta o **AnalyzeDiagramUseCase** via Composition Root.

Passo 6 - Worker executa pipeline e publica eventos

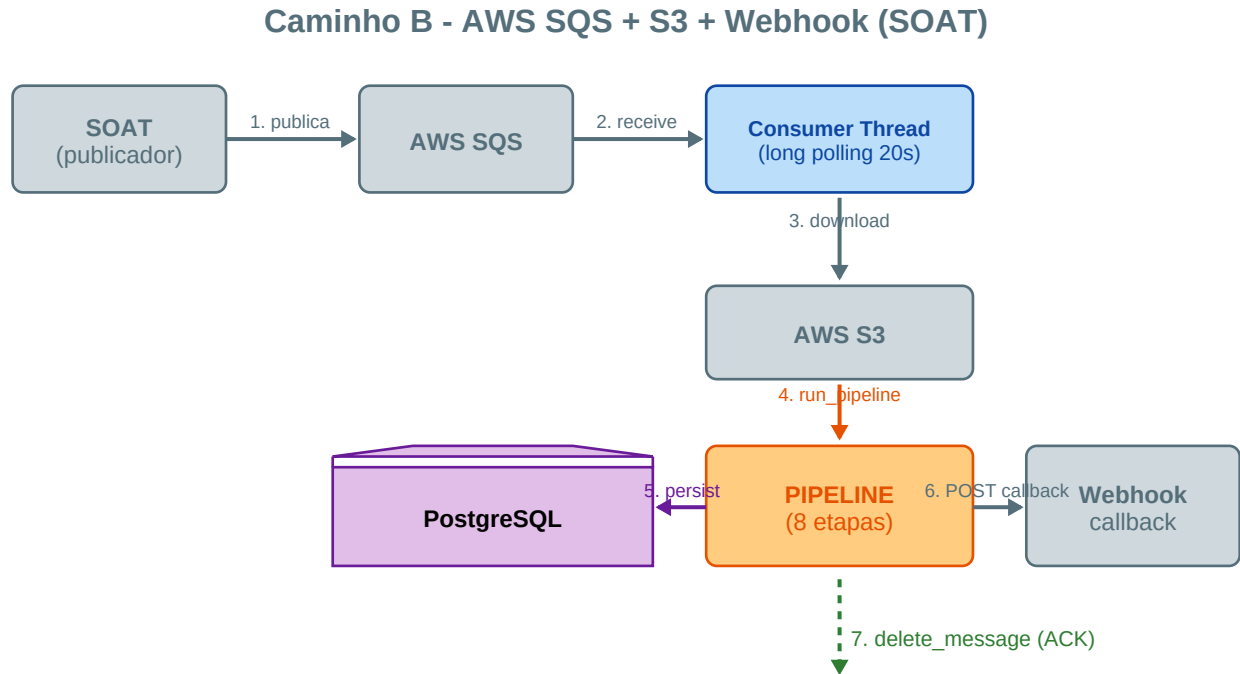
Para cada etapa, o worker publica um evento `running` antes e `done` apos concluir (com metricas: tempo decorrido, contagens). Em falha bloqueante, publica `error` e atualiza `analysis.status='erro'`.

Passo 7 - Streamlit consome e renderiza

O Streamlit atualiza a UI incrementalmente a cada evento. No evento final `step=done`, fecha o stream e renderiza o relatorio completo: sumario, componentes (grid), riscos (expanders com badges), recomendacoes (separando [RAG] das demais), score de QA e botao de download.

4. Workflow Detalhado - Caminho B (AWS SQS)

Fluxo de producao acionado pelo time SOAT. Nao exige o Streamlit - o retorno e entregue por webhook HTTP.



Retry 3x (backoff expo) no download. Webhook nao bloqueia - resultado ja persistido.

Figura: Consumer SQS com idempotencia, download do S3, pipeline e webhook

Passo 1 - SOAT publica mensagem no SQS

O SOAT faz upload no S3 e publica JSON na fila SQS com: `file_name`, `s3_url` e `callback_url`.

Passo 2 - Consumer thread recebe

Thread daemon iniciada no startup do FastAPI. Long polling com `WaitTimeSeconds=20`, `MaxMessages=5`, `VisibilityTimeout=300s`.

Passo 3 - Idempotencia e poison pill

Verifica `sqs_message_id` na tabela `analyses`: se ja existe, descarta. Se `ApproximateReceiveCount > 3`, loga warning de poison pill.

Passo 4 - Download do S3

Usa tenacity com 3 tentativas e backoff exponencial (2s -> 4s -> 10s). Falha total marca a analise como erro, envia webhook e deleta a mensagem.

Passo 5 - Execucao do pipeline

Chama `run_pipeline(file_bytes, file_name)`, que delega ao mesmo `AnalyzeDiagramUseCase`. As 8 etapas executam identicamente ao caminho A (sem Celery - a fila ja prove a assincronia).

Passo 6 - Webhook de devolutiva

POST para `callback_url` com JSON de sucesso ou erro. Retry 3x (backoff 2s/4s/8s) em 5xx e timeout; não retenta 4xx. Falha não bloqueia - o resultado já está persistido.

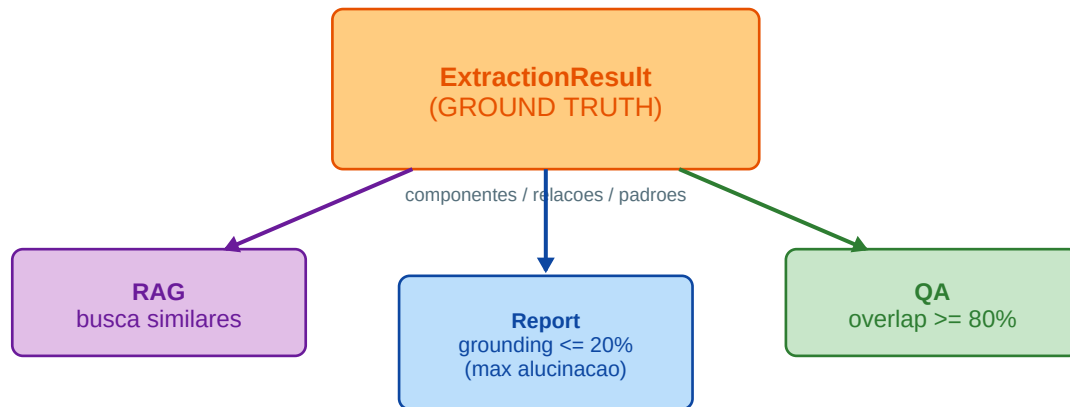
Passo 7 - Delete da mensagem no SQS

Somente após o pipeline concluir e o webhook ser tentado. Se o worker crashar antes, o SQS reentrega após o visibility timeout (at-least-once delivery).

5. Pipeline de IA - 8 Etapas Detalhadas

Executadas **sequencialmente** pelo `AnalyzeDiagramUseCase`. A saída de cada etapa alimenta a próxima. O **ExtractionResult** serve como *ground truth* para todas as validações posteriores (anti-alucinação).

Convergencia via Ground Truth



Todas as etapas são validadas contra a extração - evita alucinação.

Figura: `ExtractionResult` ancora RAG, Report e QA

5.1 Etapa 0 - Input Guardrails

Arquivo: `domain/shared/input_guardrail.py`

Sanitiza o filename (remove path traversal e caracteres perigosos), limita a 255 caracteres e detecta prompt injection (25+ padrões regex: override de instruções, role-play, exfiltração, delimitadores).

Resultado: sucesso -> prossegue; falha -> `GuardrailError` (bloqueia).

5.2 Etapa 1 - Ingestion

Valida tamanho (≤ 20 MB), detecta MIME type, valida extensão suportada (png, jpg, jpeg, gif, webp, pdf) e converte bytes -> Base64 via `DiagramFile.create()`. Cria registro em `analyses` e transiciona para *em_processamento*.

```
DiagramFile {
  file_name: "diagrama.png",
  file_type: "png",
  media_type: "image/png",
  content_base64: "iVBORw0KGgo...",
  file_size_kb: 512.3
}
```

5.3 Etapa 2 - Classification (Vision LLM)

Antes de extrair, verifica se a imagem é **realmente** um diagrama de arquitetura. O LLM recebe critérios rigorosos de aceitação (caixas, setas, componentes técnicos) e rejeição (fotos, mockups, organogramas). Exige **confidence** ≥ 0.75 .

Por que classificar antes de extrair: triagem rápida e barata evita gastar tokens em imagens irrelevantes.

5.4 Etapa 3 - Extraction (Vision LLM)

Nao usa OCR. O LLM interpreta o diagrama com compreensao semantica. System prompt define papel de *arquiteto senior* e `response_format=json_object` forca JSON valido. Retorna 4 campos obrigatorios.

```
ExtractionResult {
  components: ["API Gateway", "Auth Service", "User DB", "Redis Cache"],
  relationships: [
    "Client -> API Gateway: requisicoes HTTP",
    "API Gateway -> Auth Service: valida JWT"
  ],
  patterns: ["Microservices", "API Gateway Pattern"],
  raw_description: "Arquitetura de microservicos com..."
}
```

Persistencia: tabela `extraction_results` (JSONB).

5.5 Etapa 3.5 - Guardrail de Extracao

Apos o LLM retornar, valida o **schema** (chaves, tipos) e varre os **dados** em busca de prompt injection nos valores textuais. Limites: max 200 componentes e 500 relacionamentos.

Por que validar depois do LLM: o LLM pode devolver strings maliciosas que seriam concatenadas em prompts futuros (RAG, Report). Defesa em profundidade.

5.6 Etapa 4 - RAG (pgvector)

IMPORTANTE: non-blocking. Se falhar, retorna `RagContext.empty()` e o pipeline prossegue.

Indexacao

Cria LangChain Document com descricao + componentes + padroes + relacoes. Gera embedding com `text-embedding-3-small` (fallback `HuggingFace all-MiniLM-L6-v2`) e armazena em `langchain_pg_embedding`.

Recuperacao

`similarity_search_with_score(k=3, filter={has_report: True})`. Threshold: **score < 0.3** (distancia coseno) = > 70% similar. Se nenhum passa -> `RagContext.empty()`.

Enriquecimento

Um LLM chain sintetiza o contexto historico, identificando padroes de risco recorrentes. Esse texto e injetado no prompt da proxima etapa.

5.7 Etapa 5 - Report + Riscos (Text LLM)

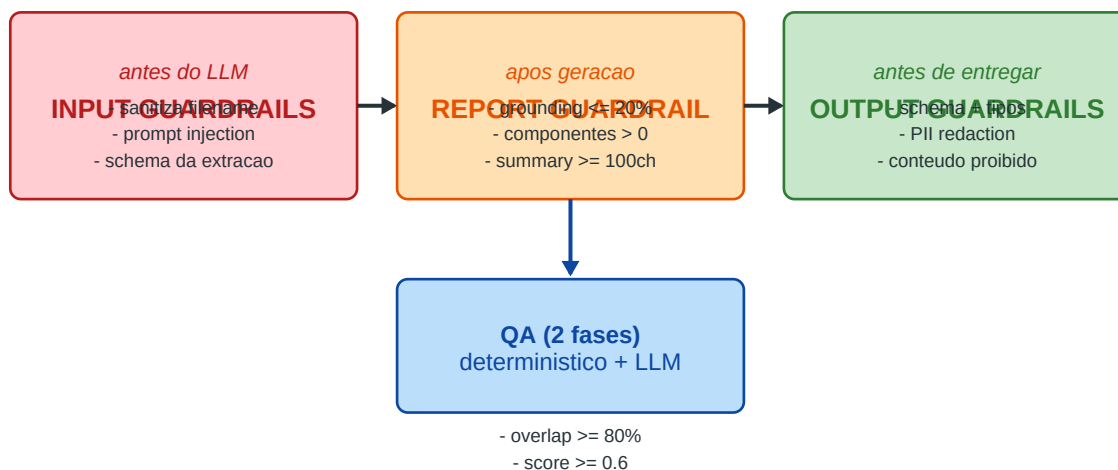
Gera o relatorio COMPLETO em uma unica chamada, incluindo classificacao de riscos. Reduz latencia e mantem coerencia entre riscos e recomendacoes. Riscos sao enums tipados em 6 categorias:

Categoria	O que avalia
SPOF	ponto unico de falha
Seguranca	auth, exposicao, endpoints
Escalabilidade	gargalos, cache, filas sem DLQ
Acoplamento	dependencias sincronas, interfaces
Observabilidade	logs, metricas, tracing
Resiliencia	circuit breaker, retry, fallback

Recomendacoes com influencia do RAG sao prefixadas com **[RAG]**. O campo `risk_severity_summary` e recalculado server-side - **nao confiamos no LLM para somar**.

5.8 Etapa 5.5 - Output Guardrails

3 Camadas de Guardrails



Defesa em profundidade: filtros multiplos, nao apenas prompt engineering.

Figura: Tres camadas de guardrails + QA, cobrindo entrada, geracao e saida

Validacoes aplicadas:

- **Schema:** chaves obrigatorias, tipos, severidades validas (ALTO/MEDIO/BAIXO)
- **Conteudo proibido:** discriminatorio, ilegal, engenharia social
- **PII:** CPF, CNPJ, email, telefone, IPv4/v6, API keys (sk-, ghp_, AWS), cartoes -> [REDACTED]
- **Grounding check:** max 20% dos componentes do relatorio podem nao existir na extracao

5.9 Etapa 6 - QA (2 fases + loop de refinamento)

Fase 1 - Deterministicas (sem LLM)

- `components_identified` nao vazio
- `architectural_risks` nao vazio
- `recommendations` nao vazio
- `executive_summary` >= 100 caracteres
- **Grounding:** >= 80% dos componentes do relatorio devem existir na extracao

Se a Fase 1 falhar, rejeita imediatamente - economia de tokens.

Fase 2 - Auditor LLM adversarial

System prompt define papel de *auditor adversarial*. Avalia com criterios ponderados:

Criterio	Peso	O que avalia
Consistencia	40%	componentes e riscos batem com a extracao
Completo	30%	campos obrigatorios preenchidos

Coerencia	20%	recomendacoes vinculadas aos riscos
Qualidade	10%	linguagem tecnica, sem generalidades

Score minimo obrigatorio: **MIN_SCORE = 0.6**.

Loop de refinamento

Se rejeitar, os issues sao enviados como feedback na proxima tentativa com secao **CORRIJA OBRIGATORIAMENTE. Maximo 2 tentativas**. Se falhar, QAError e levantada.

Resiliencia: se o LLM de QA estiver indisponivel, assume score conservador 0.7 (desde que Fase 1 tenha passado).

5.10 Etapa 7 - Persistencia + Webhook

Ao final (sucesso): INSERT em reports, UPDATE em analyses (status='analizado'), publica evento final no Redis; no caminho SQS, dispara POST `callback_url`.

6. Convergencia e Tratamento de Falhas

6.1 Ground Truth como ancora

A saída da Etapa 3 (Extraction) é a fonte de verdade. RAG, Report e QA recebem a extração original e são validados contra ela. Evita propagação de alucinações.

6.2 Bloqueante x non-blocking

Etapas BLOQUEANTES	Etapas NON-BLOCKING
0. Input Guardrails	4. RAG (fallback: RagContext.empty())
1. Ingestion	6. QA LLM indisponível (score 0.7)
2. Classification (< 75%)	7. Webhook (resultado já persistido)
3. Extraction	
3.5 Guardrail Extração	
5. Report	
5.5 Output Guardrails	
6. QA (após 2 tentativas)	

6.3 Tratamento de falhas

Em toda falha bloqueante, o sistema executa:

- `analysis.fail(step, message)` atualiza o agregado
- Repositório persiste `status='erro' + error_message`
- Evento de erro publicado via Redis (frontend informado)
- Domain Event `AnalysisFailedEvent` é emitido
- Caminho SQS: webhook com `status='erro'`
- Mensagem SQS **não** deletada - volta para retry após visibility timeout

Exceções tipadas: `IngestionError`, `ExtractionError`, `ReportGenerationError`, `RAGError`, `QAError`, `GuardrailError`. Cada uma carrega o campo `step` indicando a origem.

7. Persistencia e Observabilidade

7.1 Tabelas do PostgreSQL

Tabela	Conteudo
analyses	ciclo de vida (status, file_name, sqs_message_id, error)
extraction_results	saida do Vision LLM (components, relationships, patterns) em JSONB
reports	relatorio completo + metricas de QA
langchain_pg_embedding	embeddings (vector) + metadata, indice HNSW cosine

7.2 Redis

- **Broker do Celery** (filas de tasks)
- **Pub/Sub** de eventos SSE (canal `job:{job_id}`)
- **Event store temporario** para polling via GET `/jobs/{id}/status`

7.3 Logs estruturados (structlog)

Todos os servicos emitem JSON com timestamp ISO, nivel, logger_name e analysis_id via `logger.bind()`. Padrao do nome: **componente.acao.resultado**.

```
vision_llm.extract.done
pipeline.qa_rejected.refinement
sqs.consumer.poison_pill_detected
webhook.send.retry
```

7.4 Domain Events emitidos

Evento	Quando
DiagramReceivedEvent	analise criada
DiagramIngestedEvent	arquivo validado e convertido
ComponentsExtractedEvent	Vision LLM extraiu componentes
ReportGeneratedEvent	relatorio tecnico gerado
QAValidationCompletedEvent	QA concluiu (aprovado ou rejeitado)
AnalysisCompletedEvent	pipeline finalizou com sucesso
AnalysisFailedEvent	qualquer etapa bloqueante falhou

8. Consulta de Relatorio (report-api)

Apos o pipeline concluir, clientes consultam via report-api (:8001) - servico **read-only**. Nao passa por LLM, le direto do banco.

8.1 GET /reports (lista paginada)

```
GET http://localhost:8001/reports?limit=20&offset=0

{
  "total": 5,
  "limit": 20,
  "offset": 0,
  "items": [
    { "analysis_id": "...", "file_name": "...", "status": "analizado" }
  ]
}
```

8.2 GET /reports/{id} (detalhe)

Retorna o relatorio completo (componentes, riscos, recomendacoes, sumario, QA). 404 se nao encontrado.

8.3 GET /analyses/{id}/status (acompanhamento)

Servido pelo ia-service (nao pelo report-api). Util para clientes que nao suportam SSE fazerem polling do status.

```
GET http://localhost:8000/analyses/{analysis_id}/status

{
  "analysis_id": "uuid",
  "status": "recebido | em_processamento | analisado | erro",
  "file_name": "diagrama.png",
  "error_message": null
}
```

9. Resumo Visual do Workflow

Consolidacao de todos os fluxos em uma pagina. Use esta secao como cola de referencia rapida.

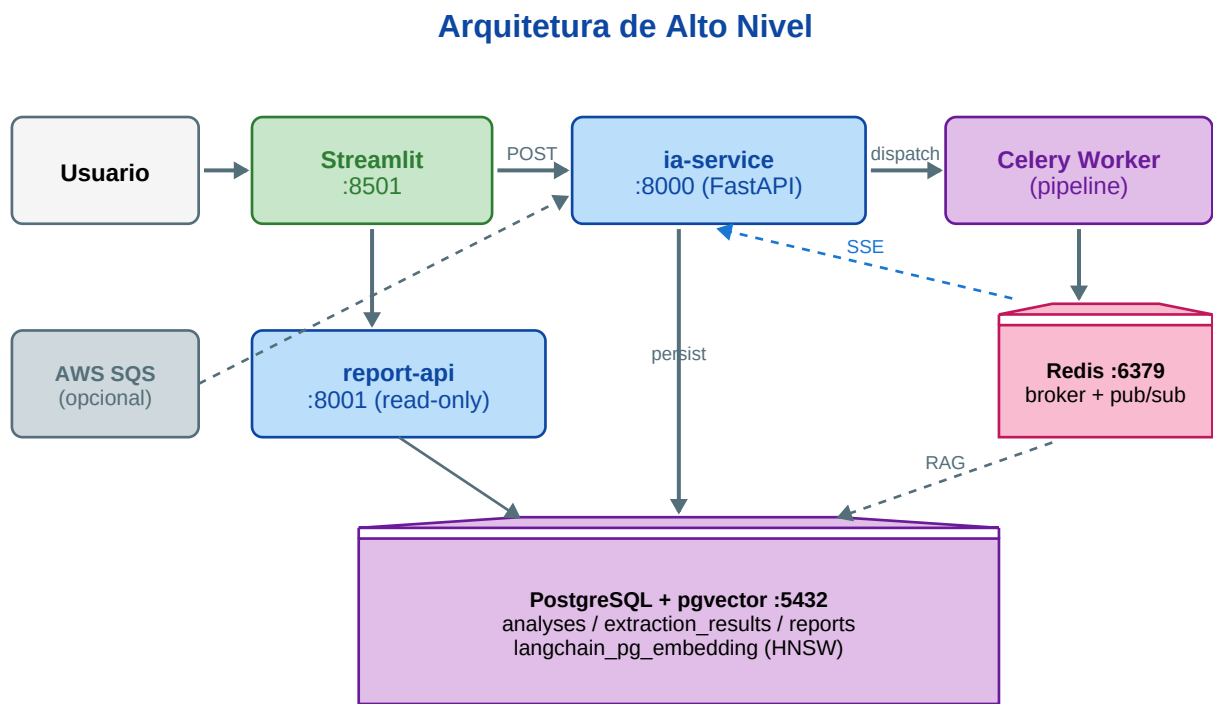
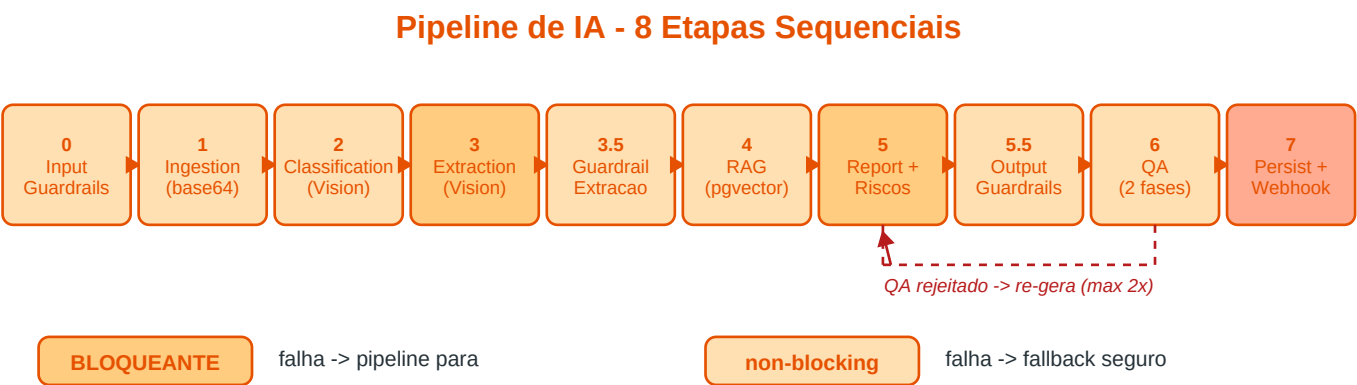


Figura: Arquitetura de alto nivel



ExtractionResult = Ground Truth

Todas as etapas seguintes sao validadas contra ele (anti-alucinacao)

Figura: Pipeline de IA com loop de refinamento

10. Glossario

Aggregate	Padrao DDD: objeto raiz que protege invariantes de um conjunto de entidades (ex: AnalysisAggregate).
Composition Root	Ponto unico onde o grafo de dependencias e montado (composition_root.py). Nenhuma outra camada pode depender dele.
Domain Event	Evento emitido pelo agregado a cada transicao relevante de estado (ex: ReportGeneratedEvent).
Grounding Check	Validacao que exige que X% dos componentes do relatorio existam na extracao original. Defesa anti-hallucinations.
Ground Truth	Fonte de verdade para validar etapas posteriores. No pipeline, e o ExtractionResult (saida do Vision API).
Guardrail	Camada de protecao para bloquear inputs maliciosos, alucinacoes ou vazamento de PII.
HNSW	Hierarchical Navigable Small World. Indice vetorial do pgvector para busca por similaridade rapida.
Non-blocking	Etapas que nao interrompem o pipeline (ex: RAG, QA LLM, webhook).
Pipeline	Sequencia ordenada das 8 etapas do AnalyzeDiagramUseCase.
Port / Adapter	Interface abstrata (Port) definida na camada de Application, com implementacao concreta (Adapter).
RAG	Retrieval-Augmented Generation. Recupera analises similares do pgvector e injeta no prompt de generacao.
SSE	Server-Sent Events. Streaming HTTP unidirecional do ia-service para o cliente (progresso em tempo real).
Value Object	Padrao DDD: objeto imutavel definido pelos seus valores (ex: DiagramFile, QAScore, AnalysisId).
Visibility Timeout	Tempo que o SQS esconde a mensagem apos o receive. Se o consumer nao deletar, a mensagem e reenviada.

Fim da documentacao. Repositorio: [hackathon-iadt](#).